

Retrofitting Pretrained Transformers with Progressive Retrieval Attention: Thin Hugging Face Adapters and a Qwen Correctness Study

Jorge Simao

EInnovator R&D – Center for the Sciences of Computation, Cognition, and Complexity

August 11, 2026

Abstract

Progressive Retrieval Attention (PRA) selects sparse, layer-native key/value (K/V) memory from references or displaced prompt history. Controlled models establish the mechanism but do not show that it can preserve a real pretrained transformer’s attention semantics. We begin a pretrained integration study with a shared PRA execution core and thin Hugging Face attention adapters. The first adapter wraps one upper layer of the frozen 596M-parameter **Qwen/Qwen3-0.6B** checkpoint. When memory is disabled, the wrapper delegates to the original module and obtains bit-exact logits, hidden states, greedy generation, and all 28 generation-cache shapes. When memory is enabled, it reuses Qwen’s pretrained projections, Q/K normalization, rotary embedding, grouped-query layout, causal mask, cache update, eager attention function, and output projection. Stored memory retains eight native K/V heads rather than expanding to 16 query heads. A CUDA smoke run materializes 22 reference tokens from CPU, generates finite text, and executes a 408-token logical prompt as a 280-token implicit head plus a 128-token direct tail with zero configured native-limit violations. These are transport and compatibility results, not retrieval-quality results. In one HotpotQA and one QASPER example, mean-gist routing selected no annotated evidence; HotpotQA failed under every tested baseline, and the QASPER answer appeared under dense context and PRA without establishing causation. The first milestone therefore supports the adapter boundary and native-KV transport, while identifying frozen zero-shot routing as the next blocking scientific problem before Llama or Gemma integration.

1 Introduction

Pretrained decoder families differ in projection layout, rotary implementation, grouped-query attention (GQA), masks, sliding windows, and generation-cache conventions. Replacing a complete model class to add memory duplicates rapidly changing library code and makes parity difficult to audit. The alternative studied here is narrower:

normal pretrained model $\rightarrow$ wrap selected attention modules $\rightarrow$ shared PRA core.

The family adapter owns only native attention semantics. The shared core owns references, chunk gists, exact routing, the materialization budget, selected K/V transfer, and metrics. This boundary follows the broader transformer interface of queries attending to keys and values [9], while preserving Qwen’s pretrained RoPE and GQA choices rather than approximating them in a parallel implementation [8, 1, 7].

This paper begins with Qwen because a sub-billion-parameter checkpoint makes exact tests practical on constrained hardware while still exercising modern RoPE and GQA. It does not yet claim cross-family portability or useful long-context QA. The contribution is a tested first adapter and an evidence hierarchy that prevents working transport from being mistaken for working retrieval.

2 PRA Recap and Prior Contracts

For layer ℓ , PRA stores reference chunks as native $K_c^{(\ell)}, V_c^{(\ell)} \in \mathbb{R}^{H_{kv} \times T_c \times d_h}$ and one or more compact gists $g_c^{(\ell)}$. A query representation $r^{(\ell)}$ scores the gists, and a global budgeter selects whole chunks. Materialized memory precedes the direct K/V sequence under one softmax:

$$\text{Attn}(Q, [K_M; K_D], [V_M; V_D]) = \text{softmax}\left(\frac{Q[K_M; K_D]^\top}{\sqrt{d_h}} + M\right) [V_M; V_D]. \quad (1)$$

This is not a separately normalized cross-attention residual. The native output projection is applied once to the combined result.

Paper 1 established controlled native-K/V routing and materialization. Paper 1.5 separated logical source position from contextual fragmentation. Continuous sources use $p_i = p_{\text{logical start}} + i$, and ordinary post-RoPE keys remain the canonical cache state. The present adapter preserves that contract by capturing each selected HF layer’s actual post-RoPE keys and by assigning source-relative positions across bounded encoding blocks.

3 Adapter Architecture

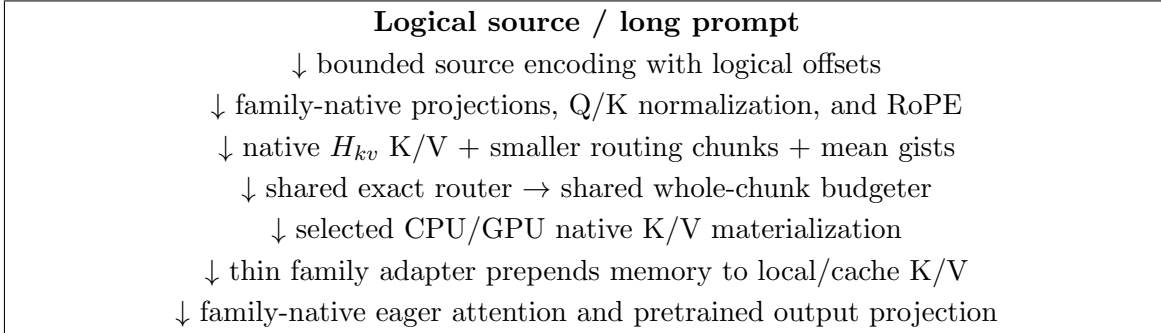


Figure 1: Paper 2 boundary. Model-family semantics stay in a thin adapter; references, routing, budgeting, materialization, residency, and metrics stay in one shared PRA core.

3.1 Shared execution core

The controlled model’s former monolithic attention forward pass was divided into reusable operations. `prepare_pra_query` selects the newest valid query and maps GQA query groups into the native K/V routing width. `route_memory` calls the exact packed-index router. `budget_selected_memory` enforces

$$L_{\text{direct}} + L_{\text{materialized}} + L_{\text{reserve}} \leq L_{\text{native, max}}. \quad (2)$$

`materialize_selected_kv` removes duplicate overlap, preserves row isolation, and moves only retained detail K/V. `combine_local_and_memory_kv` pads variable rows and prepends an additive all-visible memory mask to the family’s existing local causal mask. The controlled model and HF adapters now call this same core.

3.2 Thin HF contract

The abstract adapter exposes six family operations: project Q/K/V, apply native position encoding, normalize tensor layout, combine the native mask, invoke PRA attention, and project the output. Disabled memory takes an even narrower path: call the original attention object unchanged. This design preserves pretrained parameters and avoids checkpoint conversion.

The initial Qwen file has 227 physical lines (204 nonblank, non-comment lines). It introduces no learned parameter. Its family-specific branches distinguish Qwen2-style projections from Qwen3’s Q/K normalization; routing, budgeting, materialization, residency, and metrics remain outside the file. This is a portability measurement, not a code-minimization target.

4 Qwen Native Semantics

4.1 Projection and RoPE

The adapter calls the wrapped module’s `q_proj`, `k_proj`, and `v_proj`. Qwen3’s existing `q_norm` and `k_norm` are applied before the installed Transformers `apply_rotary_pos_emb` function. Reference capture saves post-RoPE K and position-independent V . No independent RoPE implementation is introduced.

4.2 Grouped-query attention

The tested checkpoint has $H_q = 16$, $H_{kv} = 8$, and $d_h = 128$. Cache objects retain $[B, 8, T, 128]$ K/V. For routing only, each pair of query heads sharing one K/V head is averaged, producing a vector in the same 8×128 space as a flattened key gist. K/V expansion occurs inside Qwen’s eager attention function, never in persistent PRA storage. A synthetic replay test compares this path with explicit K/V head repetition and matches within 10^{-6} .

4.3 HF cache and masks

Prefill and single-token decode both use the wrapped module’s HF `Cache.update` call. PRA memory is prepended after that update, so direct generation history appears once. A decode test with four prompt tokens and two generated tokens observes five direct cache positions at the last invocation, plus four separate memory positions. The original causal mask remains on local/cache positions; selected historical memory receives a visible prefix mask. Memory- disabled generation delegates entirely to the original module.

5 Experimental Setup

Table 1 records the first checkpoint exactly. Tests use synthetic Qwen3 configs offline. The pre-trained run uses Python 3.10.11, PyTorch 2.12.1+CUDA 12.6, Transformers 4.55.4, eager attention, FP16, and an NVIDIA GeForce GTX 950M. The frozen model is not fine-tuned.

Table 1: Pinned Qwen checkpoint and PRA placement.

Checkpoint	Qwen/Qwen3-0.6B
Revision	c1899de289a04d12100db370d81485cdf75e47ca
Parameters / layers	596,049,920 / 28
Query heads / K/V heads / head width	16 / 8 / 128
RoPE base / advertised positions	10^6 / 40,960
PRA placement	layer 27 only
Initial gist / routing policy	mean, one gist/chunk, exact top- k
Native limit / direct / selected-memory cap	256 / 128 / 64 tokens
K/V residency	CPU full cache, selected transfer

Every run writes JSON and scalar CSV with the Git SHA, model revision, software versions, device, dtype, layer selection, limits, routing settings, timings, and metrics. The exact executed commit is retained in each artifact rather than inferred from the manuscript build.

6 Adapter Parity

PRA-disabled parity is the hard gate. The baseline is evaluated first; the same in-memory model is then wrapped, leaving every parameter object in place. Table 2 reports exact results for the five-token prompt “The capital of Portugal is” and four-token greedy decode.

Table 2: Original HF model versus memory-disabled PRA wrapper.

Metric	Result
Maximum / mean absolute logit difference	0 / 0
Maximum hidden-state difference	0
Greedy token agreement	100%
Greedy sequences exactly equal	yes
Generation-cache shapes equal	all 28 layers
Finite adapted logits	yes

Single-run prefill times were 0.447 s before wrapping and 0.136 s after wrapping. This order is confounded by first-call warm-up and is not a speed comparison. Exact outputs, rather than latency, are the parity evidence.

7 Native-KV Reference Smoke

An explicit 22-token reference about Lisbon is encoded through the frozen model. The selected layer captures [1, 8, 22, 128] K/V, stores it on CPU, creates a mean gist, and transfers all 22 tokens because the source fits one routing chunk. The physical transfer is 90,112 bytes, exactly $2 \times 22 \times 8 \times 128 \times 2$ bytes for FP16 K and V. The run reports 1.6 ms routing, 0.40 ms materialization orchestration, 0.15 ms selected transfer, and 0.11 ms combined attention. These synchronized phase values are instrumentation checks on one machine, not throughput claims. Cold reference construction took 0.144 s and the warm query took 0.191 s.

The generated continuation contains “Lisbon”, but the unmodified model also knows this fact. Consequently, this observation establishes functional selected-K/V transport and generation, not a

causal retrieval benefit. The exact native-replay claim is instead restricted to the synthetic tensor test, where GQA memory plus local K/V equals explicit dense composition.

8 Implicit Head and Bounded Long Prompt

A repeated prompt tokenizes to 408 logical tokens. The direct cap retains the final 128 tokens; the first 280 become `#_head`. That head is encoded in blocks of at most 64 tokens, with source-relative positions. The direct tail receives positions 280 through 407. Every encoding operation and the final selected-memory operation stays under the 256-token configured native limit; the largest observed source-encoding call is 64 tokens and violations are zero. The final finite-logit query took 0.258 s. This proves bounded execution and offset propagation for the frozen Qwen adapter. It does not show that the router selected the information needed by an arbitrary continuation.

9 Unrestricted QA Smoke

We next run one fixed HotpotQA example [10] and one QASPER example [3]. Four conditions use the same frozen checkpoint: question-only truncation, dense text left-truncated to 256 tokens, oracle evidence text-RAG, and PRA with the question direct and the complete source in native-K/V memory. Oracle text-RAG is an upper control because it receives annotated evidence. This two-example matrix diagnoses plumbing; it cannot estimate dataset accuracy.

Table 3: Unrestricted-QA smoke outcomes. Standard normalized token F1 is shown.

Dataset	Condition	F1	Annotated evidence selected
HotpotQA	truncation; dense; oracle text-RAG; PRA	0; 0; 0; 0	PRA: 0%
QASPER	truncation; dense; oracle text-RAG; PRA	0; 0.182; 0; 0.182	PRA: 0%

HotpotQA’s expected answer is “yes”; every condition begins with “No”. QASPER’s expected answer is “no”; dense and PRA generations begin with “No”, while truncation begins with “Yes”. However, PRA routes three tail chunks, none overlapping the annotated evidence, so the QASPER output cannot be credited to retrieved evidence. Qwen also fails the oracle text-RAG control on both examples. The proper conclusion is negative: this frozen top-layer, one-gist-per-chunk configuration has not demonstrated content-causal retrieval.

The selections expose a concrete failure mode. For HotpotQA, evidence lies at token spans 107–137 and 543–588, while routed chunks lie near 1216–1318. For QASPER, evidence lies at 0–317, while selected chunks lie near 4032–4192. Post-RoPE mean-key gists and an unadapted top-layer query systematically prefer irrelevant late-source chunks in these two probes.

10 Systems Observations

The first run reaches 1,345,618,432 allocated GPU bytes, including the complete FP16 model and temporary activations. Full reference K/V remains on CPU; only selected detail K/V moves to the GPU. This is evidence that the residency path executes, not a memory-savings comparison:

no matched dense long-context peak is measured here. Likewise, cold source construction, warm routing, generation, and transfer are reported separately so cache reuse is not hidden.

The current HF path is intentionally eager and unfused. Variable memory rows are padded into a rectangle before Qwen’s native eager function. Flash attention, SDPA, custom kernels, ANN routing, asynchronous overlap, and serving concurrency remain outside this correctness phase.

11 Limitations and Next Experiments

The study has one family, one checkpoint size, one PRA layer placement, one seed for systems smoke, and two QA examples. It does not compare full dense context where the complete 1,318- or 4,207-token source fits Qwen’s advertised window, because the first experiment deliberately holds the deployment-native limit at 256. It has no matched RAG retriever, no statistical QA sample, no LoRA adaptation, and no throughput or cost study.

The next experiment should address routing before family breadth. Candidate corrections are:

1. compare pre-RoPE/content-only and post-RoPE routing gists while keeping stored detail K post-RoPE;
2. pool hidden-state or normalized pre-position keys for routing without changing native K/V materialization;
3. increase gist count and PRA layer placement only after evidence recall is measurable;
4. train a small router/gist adapter with span-level evidence supervision while freezing the pre-trained language model;
5. run enough HotpotQA and QASPER examples to estimate retrieval recall, EM, and F1 with confidence intervals, and include full dense and a real text retriever.

Only after a Qwen configuration retrieves evidence above simple controls should the same adapter contract be tested on Llama. Llama should be next in the family order; Gemma follows only after Qwen and Llama no longer require shared-core changes.

12 Related Work

PRA differs from text retrieval-augmented generation, which retrieves text and lets the model encode it in the current prompt [5]. It retrieves layer-native K/V after a separate bounded encoding path. This makes PRA complementary to RAG: a text retriever may choose documents before PRA routes chunks inside cached model state. Sparse-attention systems such as Longformer and BigBird constrain token-token connectivity inside a forward pass [2, 11]; PRA instead materializes selected state from an external addressable cache. KV-cache systems emphasize serving capacity and token retention policies [4, 12, 6]. Paper 2 asks whether selection can be inserted beneath an existing pretrained family without changing its disabled behavior.

13 Conclusion

One shared PRA core now serves both the controlled model and a thin Qwen adapter. The frozen Qwen3-0.6B checkpoint passes bit-exact disabled parity, preserves native RoPE and GQA cache layout, replays selected native K/V through generation, and executes an implicit long-prompt head

under a hard native-operation bound. These are meaningful integration results. They do not establish useful retrieval. The first unrestricted-QA probes select no annotated evidence, and even oracle text-RAG is unreliable at this model scale. The Qwen integration is generic and stable enough to continue routing work, but not yet strong enough to justify multiplying family adapters. The next milestone is content-causal evidence retrieval on Qwen; Llama follows that gate.

A Code-Level Adapter Reimplementation Guide

This appendix gives the minimum code structure needed to reproduce the integration without copying the Qwen implementation. Source references use physical line numbers at repository commit 9139ca2; the adapter source is unchanged by the manuscript-only revision containing this appendix. The line references are deliberately revision-pinned because upstream Transformers APIs and local diagnostics will move over time.

A.1 Tensor and ownership contract

A new family should add projection, position, cache, mask, kernel, and output conventions in its adapter, while leaving selection and memory policy in the shared core.

The canonical tensor contract is

$$Q \in \mathbb{R}^{B \times H_q \times T_q \times d_h}, \quad K, V \in \mathbb{R}^{B \times H_{kv} \times T_k \times d_h}. \quad (3)$$

Reference entries use $B = 1$ and keep H_{kv} , including for GQA or MQA. They are not expanded to H_q in persistent storage. The controlled replay implementation shows the only required expansion point in `memory_batching.py`, lines 216–290: after memory and local K/V are joined, native K/V heads may be repeated for the attention computation. Qwen’s installed eager kernel performs the corresponding operation in the production adapter.

A.2 Minimal forward-path skeleton

The following pseudocode is the adapter’s essential control flow. It is intentionally family-neutral; “native” operations must call the installed model implementation rather than reimplementing its mathematics.

```

01 forward(hidden, native_positions, native_mask, past):
02     if capture_is_armed:
03         q, k, v = native_project_norm_and_position(hidden)
04         save_detached_post_position_k_and_native_v(k, v)
05
06     if memory_is_disabled or reference_cache_is_empty:
07         return original_attention(hidden, native_positions,
08                                 native_mask, past)
09
10     q, k, v = native_project_norm_and_position(hidden)
11     if past is not None:
12         k, v = past.update(k, v, layer_id, native_cache_kwargs)
13     memory = shared_core.prepare_memory(q, direct_tokens=k.shape[2])
14
15     if memory.is_empty:
```

```

16         return native_attention_and_output(q, k, v, native_mask)
17     k, v, mask = shared_core.combine_local_and_memory_kv(
18         k, v, memory, native_mask, query_tokens=q.shape[2])
19     return native_attention_and_output(q, k, v, mask)

```

Lines 6–8 are a correctness boundary, not an optimization. The Qwen implementation delegates to the untouched module at `hf/qwen.py`, lines 147–158, which makes disabled behavior auditable and bit-exact. In the active path, projection and RoPE occur at lines 160–163 and `Cache.update` occurs once at lines 164–171. If routing then returns no materialized memory, lines 175–185 call the native kernel directly. Delegating at that point would execute a second cache update and duplicate generation history.

Memory is composed only after the native cache update. The shared operation at `core.py`, lines 347–389, pads variable memory rows, prepends memory K/V, creates an all-visible additive mask for valid memory positions, masks padding, and concatenates the unchanged local mask. The adapter then invokes Qwen’s eager function and one pretrained `o_proj` at `hf/qwen.py`, lines 82–100 and 197–209. Thus local and memory positions share one softmax and one output projection.

A.3 Publishing a reference

Reference construction is a bounded prefill, not a second attention mechanism. A portable implementation follows this recipe:

```

01  disable_PRA_memory_during_reference_encoding()
02  for [block_start:block_end] in bounded_encoding_blocks(source):
03      position_ids = logical_source_positions(block_start, block_end)
04      arm_post_position_kv_capture(position_ids)
05      model(block_ids, position_ids=position_ids, use_cache=False)
06      captured = consume_native_kv_for_each_PRA_layer()
07      for routing_window in overlapping_windows(captured):
08          gist = compute_routing_gists(routing_window.native_kv)
09          publish(uri, logical_offsets, routing_window.native_kv, gist)
10  restore_previous_memory_state()

```

The concrete loop is `hf/injection.py`, lines 104–167. Encoding block size limits one native model invocation; routing chunk size and overlap independently determine index granularity (lines 121–128). Each chunk slices the captured post-RoPE K/V, computes one or more small routing gists, and records logical source offsets (lines 129–160). Keeping those two scales separate is important: changing index granularity must not silently change how the pretrained model encoded the source.

The same operation implements displaced long-prompt history. At `hf/injection.py`, lines 169–181, the prefix is published under the reserved `#_head` URI and the direct tail retains continued logical position IDs. Every reference-encoding call and every direct call must satisfy the model-safe native bound even when the logical source is longer.

A.4 Porting another decoder family

A practical port begins by locating seven hooks in the installed family: the attention module inside each decoder layer; Q/K/V projections; any Q/K normalization; positional encoding; generation-cache update and its keyword arguments; native mask convention; and the eager attention plus

output projection. Implement the six abstract methods declared at `hf/adaptor_base.py`, lines 66–88, then add a narrow type-selection branch analogous to `inject_pra` at `hf/injection.py`, lines 191–211. No checkpoint conversion or parameter copy should be needed.

The recommended validation order is:

1. With memory disabled, require exact logits, hidden states, greedy tokens, and generation- cache lengths against an unwrapped copy (test lines 54–71).
2. Capture a short reference and verify post-position state, logical offsets, device residency, and H_{kv} rather than H_q storage (lines 73–92).
3. Compare GQA replay with explicit head repetition on a synthetic tensor (lines 94–117).
4. Exercise an over-limit logical prompt and verify bounded native operations plus continued position IDs (lines 119–141).
5. Generate with active memory and assert that direct cache length grows by exactly one per decoded token (lines 143–158).

Four mistakes deserve explicit tests. Applying position encoding again to stored post-position keys changes their basis. Permanently expanding GQA K/V multiplies memory without adding information. Replacing the family mask with a generic causal mask can alter padding or sliding-window behavior. Finally, calling the wrapped module after manually updating its cache appends the same local token twice. Passing transport and parity tests establishes a faithful adapter; it does not establish that the router selects causally useful evidence, which requires the separate quality controls reported in the main paper.

A.5 Revision-pinned source map

Table 4 connects the preceding control flow to the implementation.

Table 4: Source map for an independent adaptor implementation.

Responsibility	File and physical lines
Abstract family boundary	<code>src/pr_torch/hf/adaptor_base.py</code> , 14–88
Qwen projections, RoPE, native kernel, and output	<code>src/pr_torch/hf/qwen.py</code> , 43–100
Post-position reference K/V capture	<code>src/pr_torch/hf/qwen.py</code> , 102–112; <code>src/pr_torch/hf/injection.py</code> , 107–160
Disabled and active runtime paths	<code>src/pr_torch/hf/qwen.py</code> , 132–227
Routing query and GQA reduction	<code>src/pr_torch/core.py</code> , 92–121
Budget, deduplication, transfer, and materialization	<code>src/pr_torch/core.py</code> , 155–345
K/V and additive-mask composition	<code>src/pr_torch/core.py</code> , 347–389
Selected-layer injection	<code>src/pr_torch/hf/injection.py</code> , 191–211
Parity, GQA, head, cache, and selection gates	<code>tests/test_hf_integration.py</code> , 54–177

References

- [1] Joshua Ainslie, James Lee-Thorp, Michiel de Jong, Yury Zemlyanskiy, Federico Lebrón, and Sumit Sanghai. GQA: Training generalized multi-query transformer models from multi-head

- checkpoints. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pages 4895–4901, 2023.
- [2] Iz Beltagy, Matthew E. Peters, and Arman Cohan. Longformer: The long-document transformer. *arXiv preprint arXiv:2004.05150*, 2020.
 - [3] Pradeep Dasigi, Kyle Lo, Iz Beltagy, Arman Cohan, Noah A. Smith, and Matt Gardner. A dataset of information-seeking questions and answers anchored in research papers. In *Proceedings of NAACL*, 2021. Preprint: <https://arxiv.org/abs/2105.03011>.
 - [4] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with pagedattention. In *Proceedings of the ACM SIGOPS Symposium on Operating Systems Principles*, 2023. Preprint: <https://arxiv.org/abs/2309.06180>.
 - [5] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Kuttler, Mike Lewis, Wen-tau Yih, Tim Rocktaschel, Sebastian Riedel, and Douwe Kiela. Retrieval-augmented generation for knowledge-intensive nlp tasks. In *Advances in Neural Information Processing Systems*, 2020.
 - [6] Yuhong Li, Yingbing Huang, Bowen Yang, Bharat Venkitesh, Acyr Locatelli, Hanchen Ye, Tianle Cai, Patrick Lewis, and Deming Chen. SnapKV: LLM knows what you are looking for before generation. In *Advances in Neural Information Processing Systems*, 2024. Preprint: <https://arxiv.org/abs/2404.14469>.
 - [7] Qwen Team. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*, 2025.
 - [8] Jianlin Su, Yu Lu, Shengfeng Pan, Ahmed Murtadha, Bo Wen, and Yunfeng Liu. RoFormer: Enhanced transformer with rotary position embedding. *arXiv preprint arXiv:2104.09864*, 2021.
 - [9] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems*, 2017. Preprint: <https://arxiv.org/abs/1706.03762>.
 - [10] Zhilin Yang, Peng Qi, Saizheng Zhang, Yoshua Bengio, William W. Cohen, Ruslan Salakhutdinov, and Christopher D. Manning. HotpotQA: A dataset for diverse, explainable multi-hop question answering. In *Proceedings of EMNLP*, 2018. Preprint: <https://arxiv.org/abs/1809.09600>.
 - [11] Manzil Zaheer, Guru Guruganesh, Avinava Dubey, Joshua Ainslie, Chris Alberti, Santiago Ontanon, Philip Pham, Anirudh Ravula, Qifan Wang, Li Yang, and Amr Ahmed. Big bird: Transformers for longer sequences. In *Advances in Neural Information Processing Systems*, 2020.
 - [12] Zhenyu Zhang, Ying Sheng, Tianyi Zhou, Tianlong Chen, Lianmin Zheng, Ruisi Cai, Zhao Song, Yuandong Tian, Christopher Ré, Clark Barrett, Zhangyang Wang, and Beidi Chen. H2O: Heavy-hitter oracle for efficient generative inference of large language models. In *Advances in Neural Information Processing Systems*, 2023. OpenReview version: <https://openreview.net/forum?id=RkRrPp7GK0>.